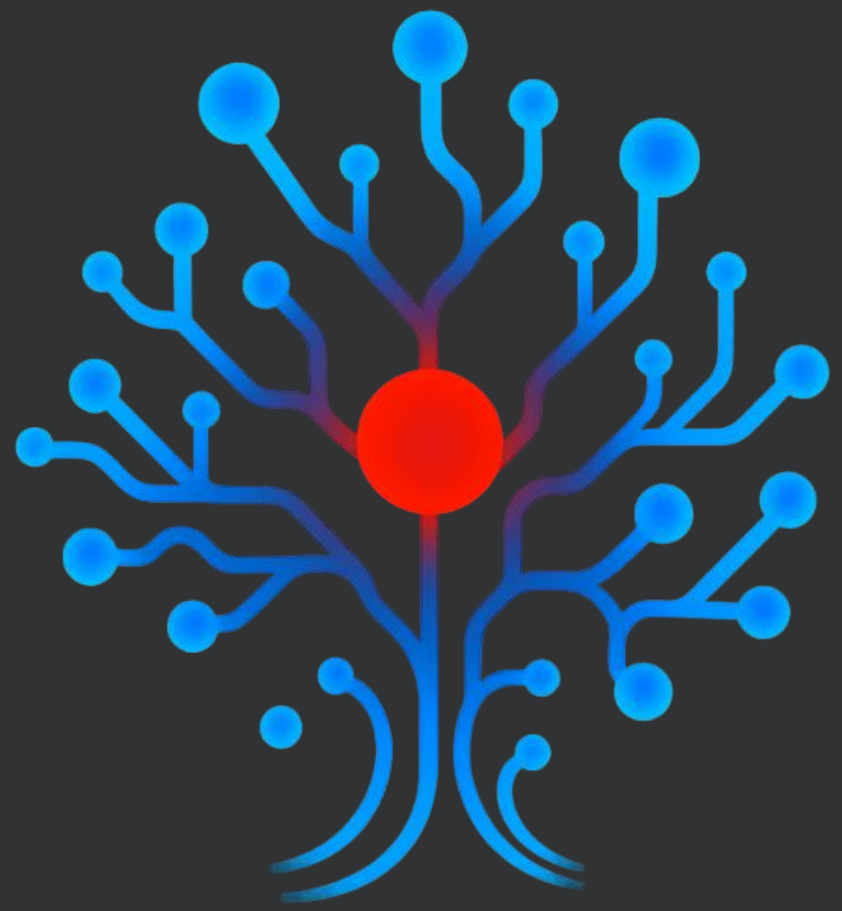
https://github.com/Raydream-Matrix-Glitch/BatVault_Memory_V2_Public



BatVault
Decision Memory V3
(Case Study)

Built end-to-end as a technical case study to explore how organisations can make decisions traceable, verifiable, and usable across workflows.

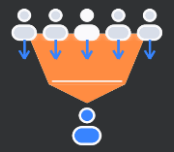
*Answers "**Why did we do X?**" with a **signed, reproducible summary** and a **policy-scoped evidence bundle***

By Christian Schaefer

BatVault

Problem and approach

This deck explores how to cut decision reconstruction time using signed receipts

Now		***BatVault***
Context sprawls; provenance & cross-links disappear		***Snapshot-bound, cross-domain context:*** *Answers are bound to a* ***point-in-time snapshot*** *and surface* ***cross-domain context*** *(same view across teams) with* ***ranked evidence you're allowed to see.***
"Why" lives in oral history; answers depend on who you ask		***Scoped by policy:*** *Self-serve rationale behind a* ***policy lens -*** *out-of-scope requests* ***fail-closed.***
Audits drag without portable proof		***Verifiable by receipt:*** *Each answer ships with a verifiable receipt (cryptographically signed) anyone can check.*

Deterministic • LLM-optional • Signed • Snapshot-bound

BatVault

System at a glance – answer & evidence

How BatVault answers a "why" question quickly with proof

One dated map that links decisions or key events to the signals and the work behind it — **policy-scoped and verifiable**

Create a trusted baseline	*Lock to a single point in time so everyone sees the same facts.*
Apply access rules	*Scope the view to the requester's policy **and mask sensitive details**; out-of-scope items don't appear.*
Highlight what matters	*Surface in-scope signals that serve **as evidence** and draft a short answer (headline, context, steps, owner, next).*
Package & sign	*Return the answer plus a receipt anyone can verify.*

Expand beyond the short answer (optional)

Consistent • Bound to a point-in-time snapshot • Access-aware • Audit-ready

Focus = one decision or event. Current scope: one hop (k=1); designed to go deeper later – see appendix

Memory returns a masked node (GET /api/enrich) or edges-only (POST /api/graph/expand_candidates); Gateway orients CAUSAL/LED_TO and signs – see appendix

BatVault

Answer, receipt, context

Example: "Why did we adopt Stripe for EU billing?"

Short answer *(from the evidence)*

(15 Aug 2025) Sam Rivera: **Adopt Stripe for EU billing**

Select and integrate Stripe to support EU billing and VAT handling for the Q4 launch

Key events:

- PCI SAQ-A scope confirmed;
- load tests met billing-service SLOs

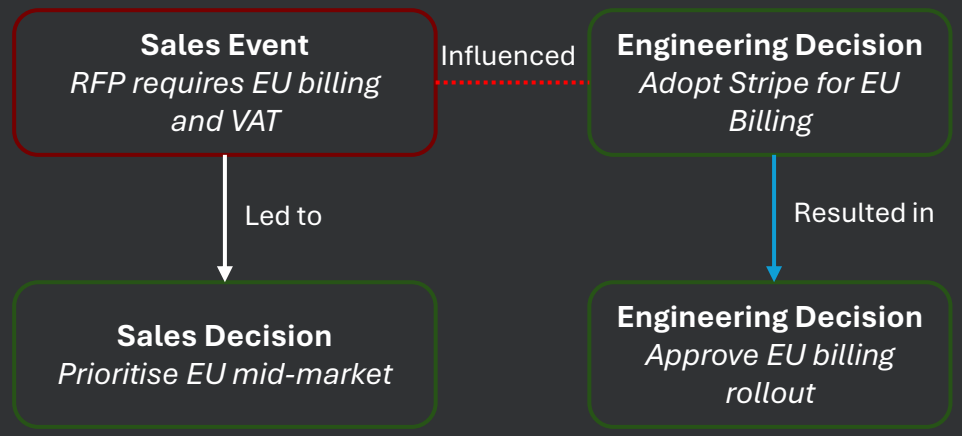
Next: Approve EU billing production rollout

Owner: Sam Rivera (VP Engineering) • **ID:** eng#d-eng-010

Proof stamp *(full fingerprints in appendix)*

What it proves / shows	Plain Text
Integrity	signed & tamper-evident
Scope	limited to what this viewer may see
Snapshot	bound to a specific point in time
Signer	signing key/owner
Timestamp	signature time

Cause & effect *(from the evidence)*



Evidence = anchor + neighbours, scoped by policy & snapshot, carried with a verifiable receipt.

BatVault

Where it plugs in

A verifiable "why" layer beside search & workflows

Sources	*Curated decisions & key events (from Jira / CRM / docs).*
Decision Memory	*Build a **signed evidence bundle** bound to a point-in-time & policy.*
Assistants / Agents	*Summaries for people - **agents create tickets with the receipt.***
Workflows	*Tasks store the receipt link. **Auditors / stakeholders** can verify in one step.*

Find → Decide & Prove → Do:
knowledge discovery (RAG) builds awareness; Decision Memory records a signed, snapshot-bound "why"; workflows carry the receipt

AI optional; deterministic fallbacks. (Appendix shows integration paths.)

BatVault

Key use cases

Owners	Predicted Outcome	Measure
Risk & Compliance	*Resolve evidence requests in one step and cut audit interruptions.*	• *Time-to-evidence (p50/p95).* • *% of audit asks closed with a receipt.*
Change & Delivery	*Faster approvals and less rework because the "why"* ***travels with the work item (human - or agent-created).***	• *Approval lead time.* • *% of work items with a valid receipt.* • *Rework rate.*
Exec & Customer Alignment	*Self-serve answers to "why did we…?" and fewer escalations.*	• *Decision-reconstruction time.* • *Number of "why" queries answered self-serve.*

BatVault

Caveats and limits

Data quality dependency	*The value tracks the quality and consistency of captured decisions / events. Gaps in intake create gaps in "why." Start with a lightweight governance loop (weekly hygiene)*
"We already have ADR / Confluence / Docs"	*A decision memory does not replace knowledge stores - it adds a **verifiable "why" layer** with receipts and scope-aware context - measured by time-to-evidence and audit closure rate.*
Integration & operating complexity	*Requires connecting Jira / Slack / CRM and operating graph, policy, cache and signing for full benefits. Mitigate via a **pilot playbook** and a narrow first-domain scope.*
Security & privacy.	*The rationale graph is sensitive. Key management, viewer scopes, and auditability matter (see next-steps hardening).*
Change management	*Early scepticism and "access denied" friction - use human-in-the-loop reviews and explicit escalation paths.*

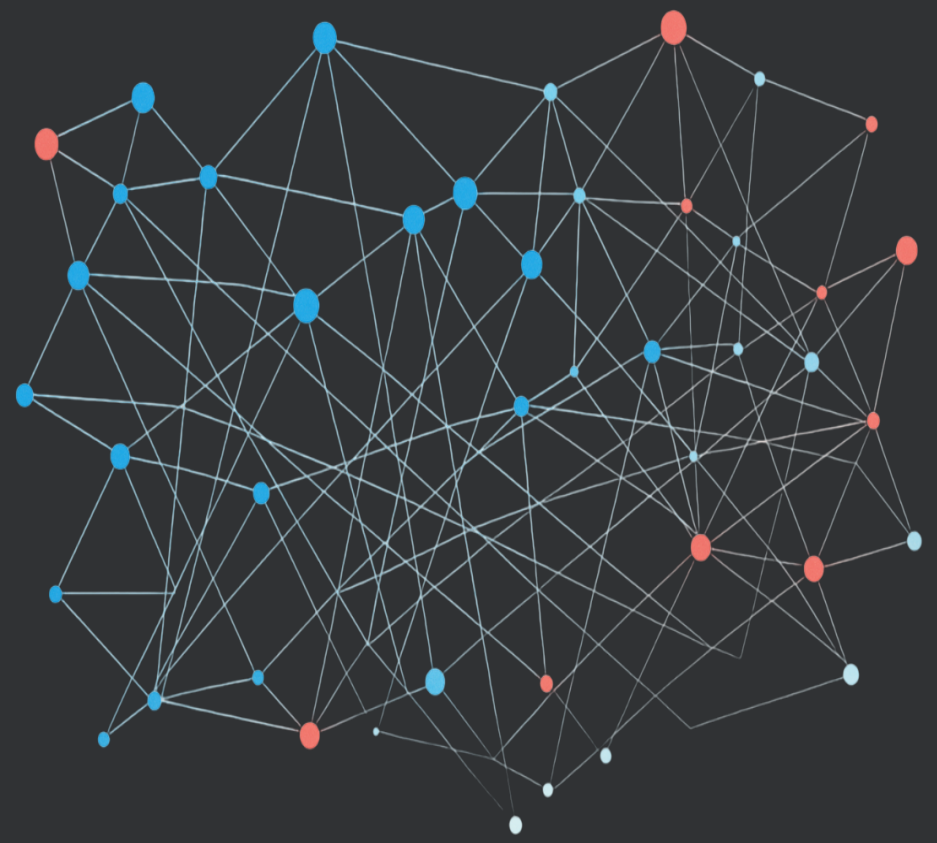
What’s next – scaling BatVault

Security hardening
Service-to-service authentication. Store signing keys in KMS with rotation. Integrate an identity provider for fine-grained access control.

Simple agent integration
Add a thin layer that forwards completed receipts into Jira issues.

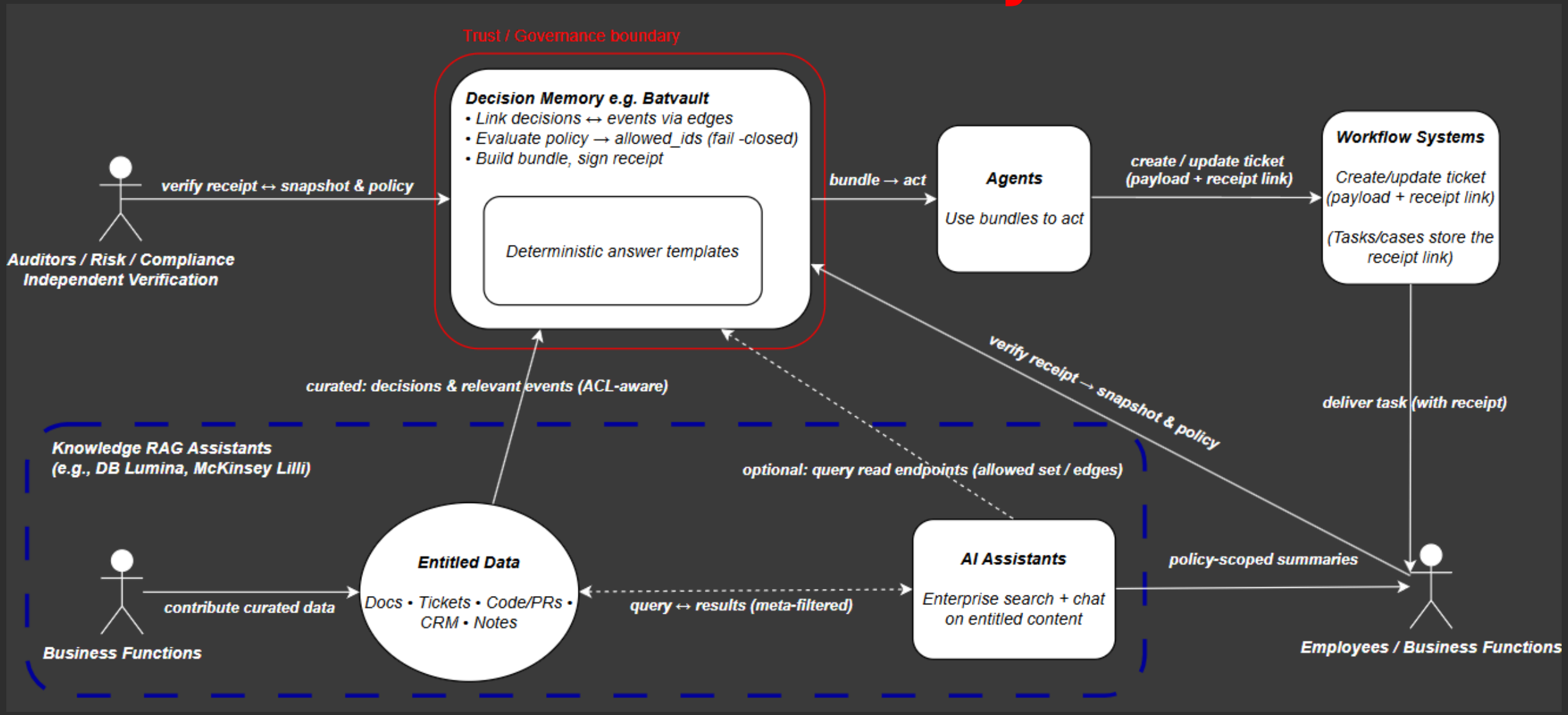
Frontend & schemas
Interactive graph UI. Extend schemas to support multi-hops and provide actionable executive dashboards (for example, time-to-decision).

Pragmatic AI use
Deploy AI (at the edges) for intent classification & retrieval with deterministic fallbacks.



APPENDIX

Decision Memories – where they fit

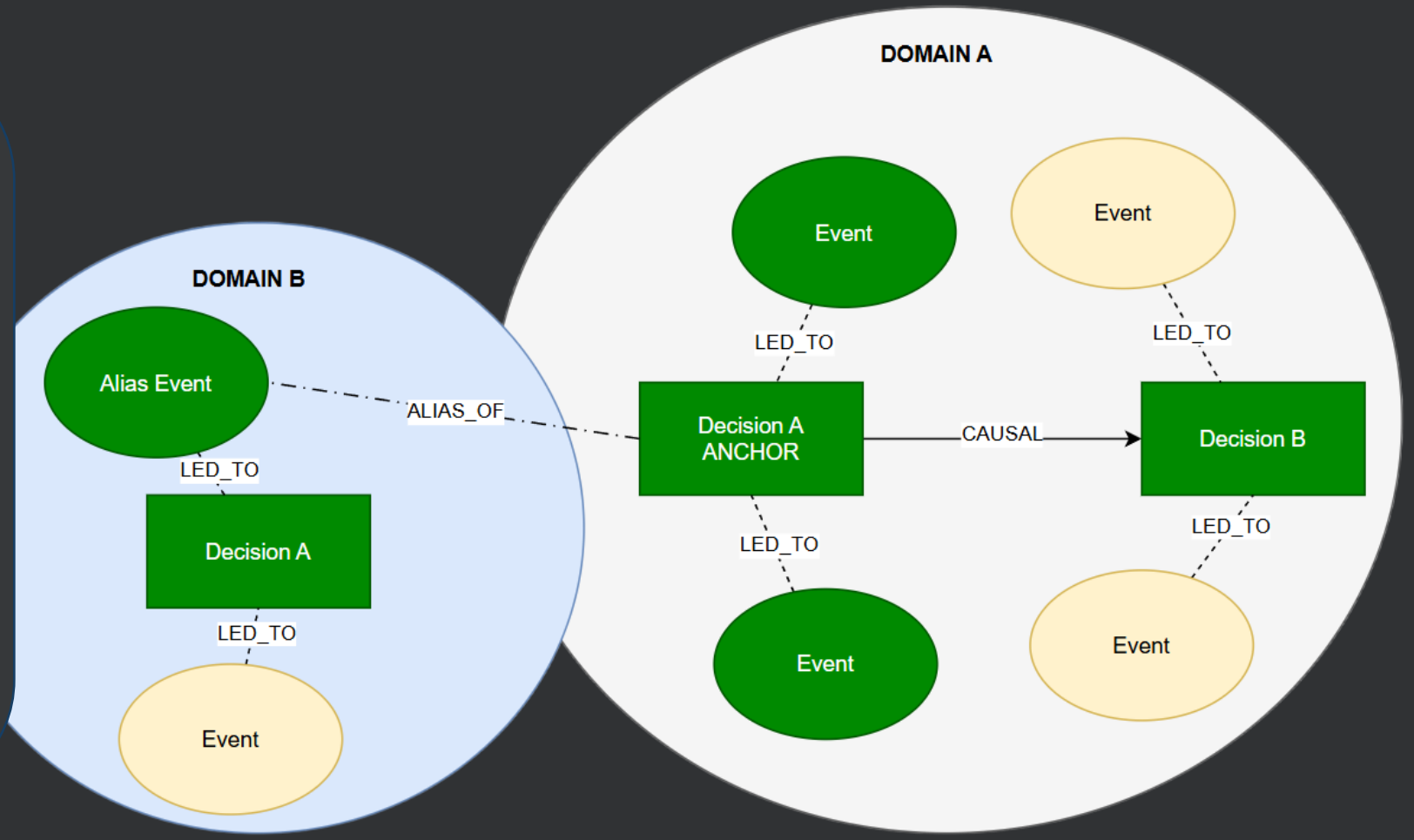


LLM use optional – always provide answers using deterministic fallbacks and templating.
Bundle = anchor + edges + answer + meta {policy_fp, allowed_ids_fp, snapshot_etag, bundle_fp}.

Data model – why-decision-resp. schema

Choose a starting point and gather surrounding evidence to produce the answer

Component	***Purpose***
Nodes	
Decisions	*what*
Events	*why*
Anchor	*starting point* *(everything is relative to the anchor)*
Relationships (Edges)	
CAUSAL	*follow-up decision*
LED_TO	*signals / drivers*
ALIAS_OF	*cross-system alias from event to decision in another domain*



ALIAS_OF neutral; orientation in Gateway service

the green components represent returns as part of the current BatVault response schema

BatVault

Micro-model - opinionated example

Minimal mandatory fields, domain-scoped anchors, canonical UTC timestamps, neutral edges, x-extra for extensions

Decision (R)	id (storage key; anchor = domain#id), type="DECISION", title, decision_maker{id, role}, domain, timestamp RFC-3339 UTC
Decision (O)	x-extra (opaque JSON)
Event (R)	id, type="EVENT", title, domain, timestamp (RFC-3339 UTC)
Event (O)	x-extra, decision_ref (points to a decision & **auto-derives an ALIAS_OF edge from decision_ref**)
Edge (R)	id (derived from type/from/to), type ∈ {LED_TO, CAUSAL, ALIAS_OF}, from, to, timestamp
Edge (O)	domain (storage domain)

O = Optional; R = Required

Schema fields would vary from org to org depending on needs forming a core ontology and domain-specifics

BatVault

Key terms & fingerprints

snapshot_etag	• Provided/persisted by ingest, stored in meta, mirrored on every read. • Binds answers to a point in time. Used in evidence and bundle cache keys.
ETag / X-Snapshot-ETag (Header contracts)	• HEAD /api/enrich - request: If-None-Match: "<etag>" (quoted). Response: ETag: "<etag>" (quoted) and x-snapshot-etag: <etag> (unquoted). 304 on match, 200 otherwise. • Data endpoints - precondition via header X-Snapshot-ETag: <etag> (unquoted) or body snapshot_etag: "<etag>" (JSON string; value equals the same unquoted token). 412 on missing/mismatch. • Responses - mirror x-snapshot-etag: <etag> (unquoted). Header names are case-insensitive; canonical forms: request X-Snapshot-ETag, response x-snapshot-etag.
graph_fp	• Deterministic fingerprint of the edges this viewer was allowed to see at this snapshot. • Anchor k=1 **plus alias tail**: inbound ALIAS_OF (stored as event → anchor) **then one outbound hop** to decisions **(max 3)** in the alias event's domain. • Apply ACL and filtering, then dedupe, sort (type, from, to, timestamp), and hash. • Memory computes graph_fp over ACL-clamped edges at the snapshot and returns it in meta.fingerprints.graph_fp.
allowed_ids_fp & policy_fp	• Fingerprints of the visibility set and the effective policy. Prevents cache bleed across roles. • If a caller sends x-policy-key that doesn't match the computed policy_fp, the request proceeds and a structured policy_fp_mismatch audit log is emitted. X-Policy-Advice may be returned.
bundle_fp	• Public fingerprint of the final bundle. Lives at response.meta.bundle_fp and equals receipt.covered. Used in cache keys (not used in artifact filenames).
requested_ids & cited_ids	• requested_ids are the IDs the caller asked to enrich. They must be a subset of allowed_ids or the request fails. • cited_ids are the IDs referenced in the answer. They are always a subset of allowed_ids.
schema_fp	• Fingerprint of the shipped JSON schemas across Memory and Gateway. Returned as **X-BV-Schema-FP.** • Used to version UIs and caches so schema drifts do not break clients.

Schema FP is computed in both Gateway and Memory. Values are expected to match. It is exposed only as the response header **X-BV-Schema-FP** on JSON responses and not on HEAD. It is not included in /config or in response bodies, so the frontend does not consume it at the time of writing (planned)

BatVault

Write path

Normalisation:
Upper-case types. Use RFC-3339 UTC-Z timestamps. Validate edge anchors and decision_ref.

Build ALIAS_OF edges:
For each EVENT.decision_ref, build event → decision with event timestamp and domain. Compute deterministic edge.id. Collect rejects.

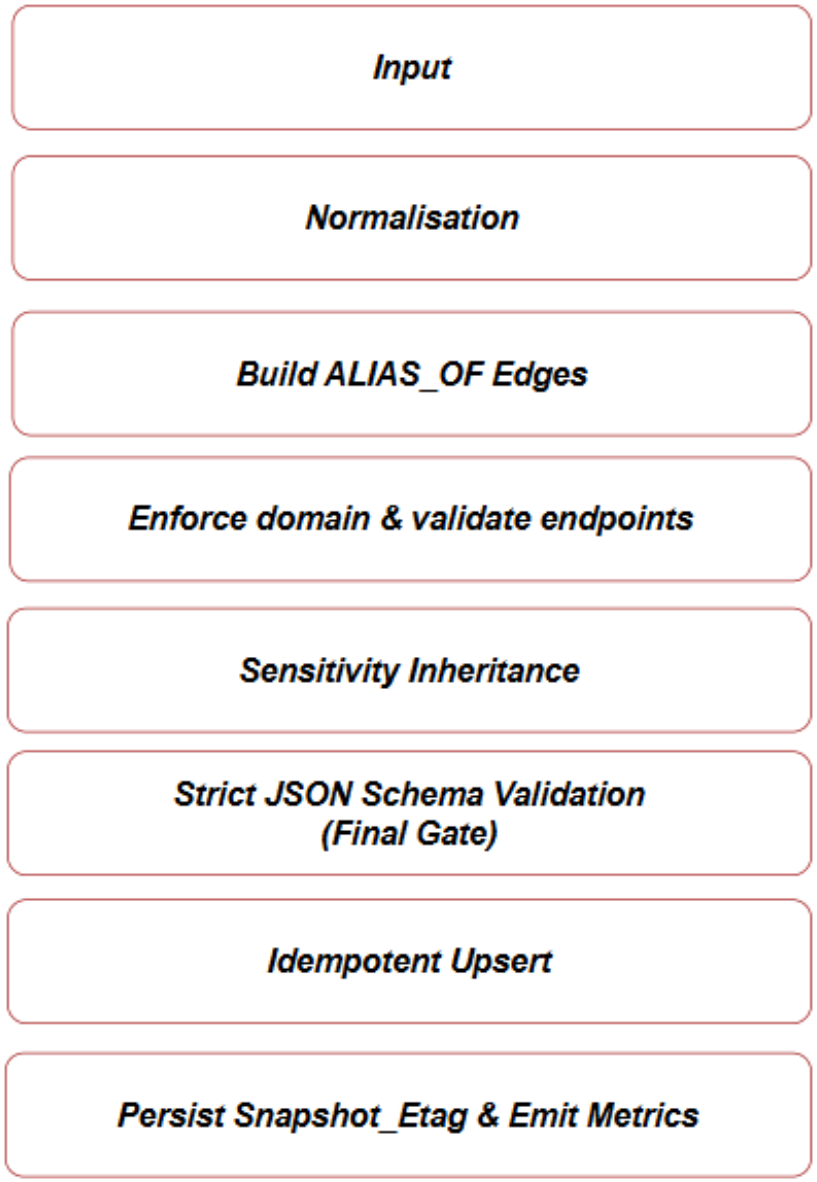
Enforce domain policy & validate endpoints:
LED_TO / CAUSAL must be same-domain. If ALIAS_OF.domain exists, it must equal the event domain. Verify anchors exist in the batch.

Sensitivity inheritance:
If event sensitivity is missing, inherit the most-restrictive level from connected decisions. Record in x-extra.sensitivity_inheritance.

Strict JSON schema validation:
Validate all nodes and edges. Aggregate errors. Fail-closed.

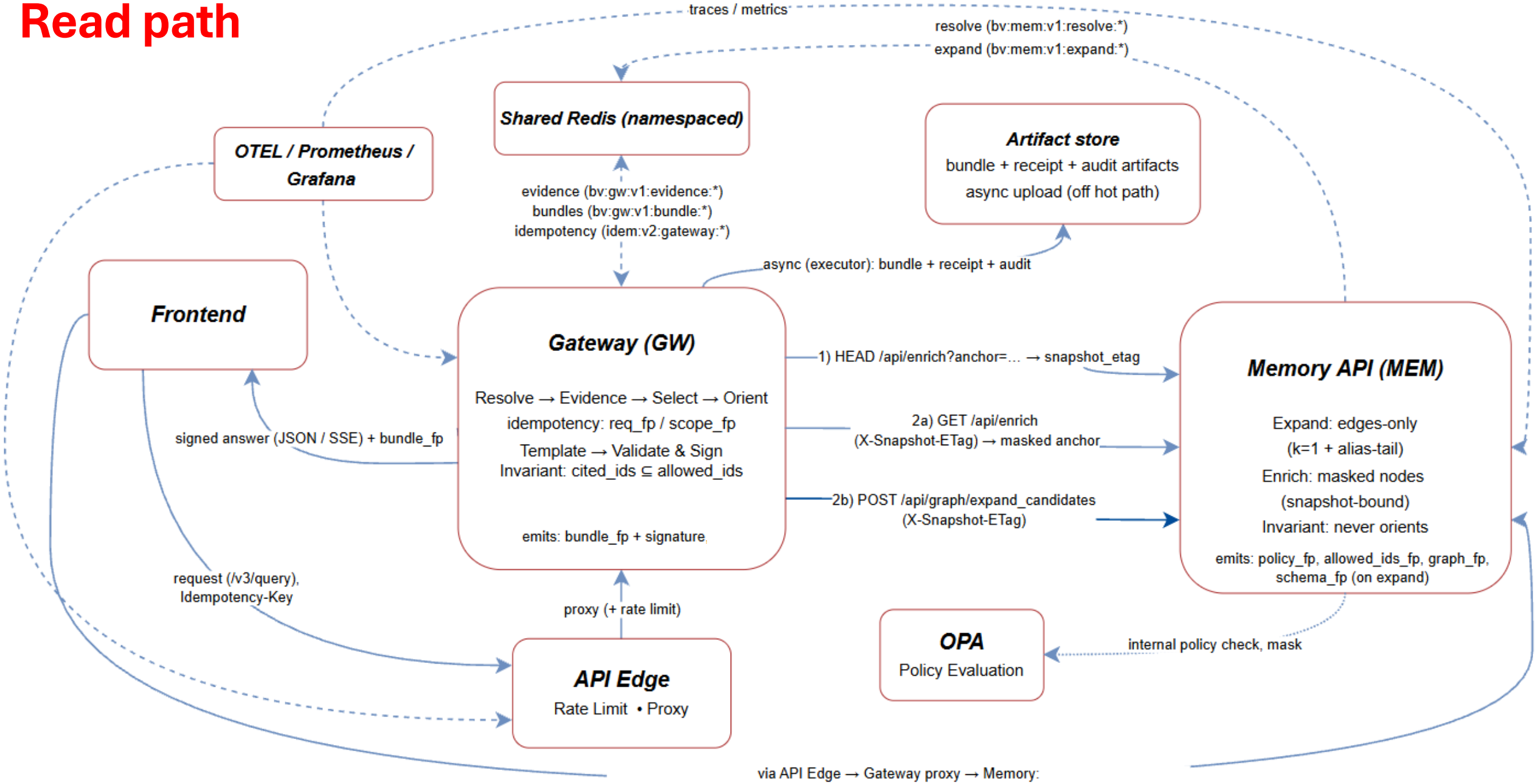
Idempotent upsert:
Upsert validated nodes and edges into the store. Deterministic IDs ensure idempotency. Return counts of written and rejected items. Use micro-batch with retry and jitter. Fallback per document on repeated failure.

Persist snapshot_etag & emit metrics:
Store snapshot_etag for point-in-time reads. Emit write, alias, and inheritance counters.



stale records are pruned prior to upsert using the **previous snapshot_etag**

Read path



Ingest stage persists the new snapshot_etag to the store. Memory echoes it on every read and requires it on snapshot-bound requests

Solid = data
Dashed = telemetry / cache / idempotency

Read path – endpoints & caches (V3)

Invariants	
Snapshot-bound	callers must send X-Snapshot-ETag on snapshot-bound reads (else 412)
Policy-scoped	requested_ids ⊆ allowed_ids; enforced in Gateway and re-enforced in Memory
Fingerprints	responses mirror X-BV-Policy-Fingerprint; graph responses also mirror X-BV-Graph-FP

Caches (Redis Namespaces)	
Memory	bv:mem:v1:resolve:*, bv:mem:v1:expand:*
Gateway	bv:gw:v1:evidence:*, bv:gw:v1:bundle:*

Public fingerprints are SHA-256 over canonical JSON (prefixed "sha256:<hex>").
Redis cache keys use BLAKE2s with a 10-byte digest.

Errors	
412	Missing or mismatched snapshot
403/404	403/404 Policy deny (status is selected by server policy; default 403, optionally 404)
304	HEAD with matching **If-None-Match**
429	Load shedding (includes **Retry-After**)

Happy Path	
GW	HEAD /api/enrich and take x-snapshot-etag (or strip quotes from ETag)
GW	Data calls with **X-Snapshot-ETag** (enrich and expand)
MEM	Return masked node or edges-only and mirror fingerprints
GW	Orient edges and sign. Compute meta.bundle_fp.
GW	Respond (JSON or final SSE) and mirror fingerprints
Async	Upload bundles to MinIO and populate caches

BatVault

API Surface V3 – Memory API & Gateway

Memory API		
HEAD	/api/enrich?anchor=	HEAD /api/enrich?anchor= Freshness probe. Returns ETag (quoted) and x-snapshot-etag (unquoted). Sends 304 when If-None-Match matches.
GET	/api/enrich?anchor=	Masked node; requires X-Snapshot-ETag; mirrors x-snapshot-etag, X-BV-Policy-Fingerprint.
GET	/api/enrich/event?anchor=	Event-only enrich. Same preconditions and headers as above.
POST	/api/enrich/batch	Batch masked nodes. Batch masked nodes. Requires snapshot precondition (X-Snapshot-ETag header or body field.
POST	/api/resolve/text	Text → candidate anchors. No snapshot precondition. Returns { matches[], resolved_id? }.
POST	/api/graph/expand_candidates	Edges-only (k=1); requires X-Snapshot-ETag; returns graph.edges[], meta.allowed_ids_fp, policy_fp; sets X-BV-Graph-FP.

Gateway		
POST	/v3/query	Main “Why?” (JSON or SSE); supports Idempotency-Key; response meta includes bundle_fp, allowed_ids_fp, policy_fp, snapshot_etag
GET	/config	FE bootstrap: base URLs, endpoints map, timeouts, signing public key.
POST	v3/verify v3/verify/upload	Verify signed receipt. Verify signed receipt (upload).
GET	/v3/bundles/{request_id}?name=bundle_view|bundle_full /v3/bundles/{request_id}/download?name=bundle_view|bundle_full /v3/bundles/{request_id}/receipt.json	Streams JSON artifacts (view is filtered by schema). Returns presigned URL. When not yet available, returns 202 with Retry-After and a safe same-origin fallback. Raw receipt (also HEAD).

For header contracts and fingerprints, see “Key terms & fingerprints”

BatVault

Frontend – thin, typed, and snapshot aware

Typed SDKs from source-of-truth specs	OpenAPI (gateway.json, memory.json) → generated clients & React hooks; JSON-Schema → TS types. Artifacts are **committed** and checked in CI.
Streaming "/query"	ND-JSON / SSE with **Idempotency-Key**. Streams **expose policy fingerprints & request IDs -** the client adopts them for follow-ups (no auto-retry).
Snapshot preconditions in UI	Snapshot-bound calls send the server's "X-Snapshot-ETag".
Offline-friendly code-gen	Scripts allow **offline regeneration -** FE boots by default with pre-generated artifacts.
Coupling that de-risks change	When schemas change, types break at compile time - no silent drift between frontend / backend.

Code gen & governance

Guardrails & governance	
JSON-Schema	packages/core_models/src/core_models/schemas/*.json
OpenAPI	batvault_frontend/openapi/{gateway.json,memory.json} (via ensure_codegen_venv.sh → dump_openapi.py)
Policy	Generated specs and types are committed so reviewers diff the **contract**, not tool output.

Generated artifacts	
Backend	Pydantic models from JSON-Schema in packages/core_models_gen/** *(a small temporary shim remains and is being replaced)*
Frontend	TypeScript types in batvault_frontend/src/types/generated/**
Typed clients & hooks (OpenAPI → **orval**)	src/sdk/src/sdk/{gateway.client.ts,memory.client.ts}. Thin wrapper src/sdk/client.ts adds **Idempotency-Key**, ND-JSON/SSE helpers, and snapshot/policy header helpers
Ergonomics	**/v3/query** streams (JSON or ND-JSON / SSE). Reads are snapshot-aware

CI enforcement & drift control	
Spec-drift checks	scripts/check_codegen_sync.sh (fails when generated files are out of date)
Breaking-change scanners	scripts/schema_compat_check.py (requires /type/ or rename changes to be explicit)
Versioning & ownership	scripts/codegen_schemas.sh (deterministic generation)

Schemas are the contract; clients & validators are generated, committed, and verified in CI. No silent drift

Artifacts

Bundles	Description / Content
View Bundle (minimal, public)	response.json, receipt.json, trace.json, bundle.manifest.json.
Full Bundle (debug)	View items plus evidence_pre.json, evidence_canonical.json, evidence_post.json, gateway.plan.json, validator_report.json.

Artifact layout (object storage)	Description / Content
response.json (signed) (View + Full)	Signed envelope — schema version + final oriented answer + graph edges + meta with allowed_ids_fp, policy_fp, snapshot_etag, bundle_fp. Everything else describes or validates this.
gateway.plan.json (Full)	Deterministic plan { selector_policy_id, truncation_action, budgets { max_edges, max_events, timeout_ms } } used to build the bundle.
evidence_pre.json (Full)	Evidence snapshot as collected from Memory/expand
evidence_post.json (Full)	Evidence after final processing (selection and orientation) that fed the answer.
validator_report.json (Full)	Results of schema and invariant checks over the view bundle (response.json, trace.json, bundle.manifest.json, receipt.json).
bundle.manifest.json (View + Full)	Deterministic inventory of artifacts { name, sha256, bytes, content_type }.
receipt.json (View + Full)	Signature block { alg: ed25519, key_id, sig, covered, signed_at }. Covered equals meta.bundle_fp of response.json.
evidence_canonical.json (Full)	Evidence after normalization (dedupe and clamp). Debugging aid. Not part of the signed view.
trace.json (View + Full)	Compact execution trace - request summary, gateway path, response preview, validator summary.

Signing and Verification

Gateway (signs)
Build the **public response**
Compute **covered**: canonical JSON of the response **without** meta.bundle_fp (sorted keys, no µs) → SHA-256 → prefix sha256:<hex>.
Sign covered with Ed25519 (env-provided key) → emit receipt.json { alg:"ed25519", key_id:"gateway/k1", sig:<base64>, covered:"sha256:<hex>", signed_at:"...Z" }.
Mirror the digest to meta.bundle_fp and assert meta.bundle_fp == receipt.covered.
Fail-closed if the signer isn't configured (no unsigned receipts).

Key distribution
Clients fetch the gateway public key from GET /config → signing.public_key_b64 (or /keys/gateway_ed25519_pub.{b64,pem}); if GATEWAY_SIGNING_REQUIRED=1 and no key is configured, /config returns 503.

Clients (verify)
Recompute computed = sha256(canonical_json(envelope (schema_version + response) minus meta.bundle_fp)) → sha256:<hex>.
Check response.meta.bundle_fp === computed and receipt.covered === computed.
Verify receipt.sig with **Ed25519** using the gateway public key.

Canonicalization: sort keys, drop microseconds; compute sha256:<hex> over the canonical envelope (response without meta.bundle_fp)
Anyone can verify a bundle with the public key; bundle_fp never lives inside meta.fingerprints (it's only in meta, mirrored from the receipt)

BatVault

Key design decisions

Verifiable-by-design receipts	Gateway signs the canonical envelope (canonical JSON → SHA-256 → Ed25519). meta.bundle_fp == receipt.covered. (Header details: see "Key terms & fingerprints")	services/gateway/src/gateway/builder.py services/gateway/src/gateway/signing.py packages/core_utils/src/core_utils/fingerprints.py
Snapshot-bound reads (fail-closed)	Snapshot-bound: data calls require X-Snapshot-ETag (else 412); HEAD /api/enrich" uses quoted ETag + If-None-Match. (See “Key terms & fingerprints”)	services/memory_api/src/memory_api/app.py
Minimal disclosure: Memory emits edges-only; Gateway orients & signs	Memory returns a masked node for /api/enrich and edges-only for /api/graph/expand_candidates. Gateway orients CAUSAL / LED_TO and signs the public envelope.	packages/core_models/src/core_models/graph_view.py services/gateway/src/gateway/orientation.py services/gateway/src/gateway/builder.py
Policy-first & fail-closed	Server recomputes effective policy; policy_fp fingerprints it. Extra_visible fields allow-listed by policy.	services/memory_api/src/memory_api/policy.py packages/core_policy_opa/src/core_policy_opa/adapter.py
Scope & audit invariants	JSON mirrors: X-BV-Policy-Fingerprint; expand/batch also X-BV-Allowed-Ids-FP; graph responses add X-BV-Graph-FP. On X-Policy-Key mismatch: proceed + policy_fp_mismatch audit; optional X-Policy-Advice.	services/memory_api/src/memory_api/app.py services/memory_api/src/memory_api/policy.py
Domain & alias rules	Alias tail (one hop, cap three) only for CAUSAL / LED_TO; ALIAS_OF.domain must match the alias event domain.	services/ingest/src/ingest/pipeline/graph_upsert.py services/memory_api/src/memory_api/app.py

Behaviour is policy-driven: server computes policy_fp. If "X-Policy-Key" differs, request proceeds; policy_fp_mismatch is logged.

BatVault

Key design decisions (2)

Write-path correctness	Validate nodes and edges against JSON-Schema. Derive deterministic edge IDs. Enforce existence checks and sensitivity inheritance (events inherit the most-restrictive sensitivity from connected decisions, recorded in x-extra).	packages/core_validator/src/core_validator/validator.py services/ingest/src/ingest/pipeline/graph_upsert.py
Deterministic fingerprints & cache keys	graph_fp, allowed_ids_fp, policy_fp, and public bundle_fp come from canonical JSON. Public fingerprints use SHA-256. Redis keys use privacy-safe BLAKE2 over those inputs.	packages/core_utils/src/core_utils/fingerprints.py packages/core_cache/src/core_cache/keys.py
Budget gate & stage timeouts	Deterministic budget gate with per-stage timeouts (resolve, expand, enrich, validate) to bound requests.	services/gateway/src/gateway/budget_gate.py, packages/core_config/src/core_config/constants.py packages/core_http/src/core_http/client.py
Idempotency & safe retries	Idempotency-Key is hashed to a Redis record with TTL. We merge progress deterministically so retries are safe.	packages/core_idem/src/core_idem/__init__.py
Observability & context propagation	OTEL bootstrap is optional. x-trace-id is always present. Outbound HTTP injects W3C Trace Context. Error handlers standardize failure modes.	packages/core_observability/src/core_observability/otel.py packages/core_http/src/core_http/client.py packages/core_http/src/core_http/errors.py
Schema fingerprint header	Responses include X-BV-Schema-FP to version caches and UIs. It appears on JSON responses, not on HEAD.	services/memory_api/src/memory_api/app.py services/gateway/src/gateway/app.py

bundle_fp location: see “Signing and Verification”

Schema FP is computed in both Gateway and Memory. Values are expected to match. It is exposed only as the response header **X-BV-Schema-FP** on JSON responses and not on HEAD. It is not included in /config or in response bodies, so the frontend does not consume it today (planned)

BatVault

Key platform guarantees

Predictable first paint	Deterministic fingerprints (graph_fp, allowed_ids_fp, policy_fp, bundle_fp) drive evidence and bundle caches, so identical inputs return fast and consistently.
Policy-enforced scope	Memory recomputes allowed_ids; /api/enrich ACL-checks anchors; /api/enrich/batch fails closed on any out-of-scope ID (snapshot-bound). Gateway never widens scope; cited_ids ⊆ allowed_ids.
Verifiable receipts	Gateway /v3/query JSON responses include a public bundle_fp and an Ed25519 signature over the canonical signed envelope.
Deterministic & idempotent	Canonical JSON and deterministic ID derivation produce the same bundle for the same inputs. The view shaper and Gateway deduplicate edges to avoid double counting. Idempotency-Key merges progress to make safe retries deterministic.
Controlled latency & fail-closed	Per-stage timeouts - resolve, expand, enrich, validate - and strict validation ensure errors return quickly and partial answers are never served.

Metrics & KPIs to watch

Product

Time to answer (end-to-end)
*p50 / **p95***

***First paint latency** (Memory expand)*
p50 / p95

Decision throughput
 # answers/week (or # teams using receipts)

Platform

***Bundle build time** (Gateway) - p50 / p95*

***Cache hit rate** - evidence/bundles*

***% signed bundles** (signature configured)*

***Policy boundary signals** - 403/404 rate*

***Snapshot drift** - 412 rate (precondition failed)*

***Enrich batch size** - median / p95 (predictable costs)*

System Screenshots and End-to-End Integrity

https://github.com/Raydream-Matrix-Glitch/BatVault_Memory_V2_Public

End-to-end integrity

RID c9c22e20d96b4a69

```
# Vars (bash)
HOST=http://127.0.0.1:8081
RID=c9c22e20d96b4a69
H=(-H 'X-User-Id: debug' -H 'X-User-Roles: ceo' -H 'X-Policy-Key: dev-default' -H 'X-Policy-Version: 0')

# 1) Snapshot + View → files → proof
SNAP=$(curl -sI "$HOST/memory/api/enrich?anchor=eng%23d-eng-010" "${H[@]}" \
  | awk -F': ' 'tolower($1)=="x-snapshot-etag"{print $2}' | tr -d '\r'); \
curl -sS "$HOST/v3/bundles/$RID?name=bundle_view" "${H[@]}" -H "X-Snapshot-ETag: $SNAP" -o
bv.json; \
jq -r '."response.json"' bv.json > response.json; jq -r '."receipt.json"' bv.json > receipt.json; jq -r
'."trace.json"' bv.json > trace.json; \
[ "$(jq -r '.response.meta.bundle_fp' response.json)" = "$(jq -r '.covered' receipt.json)" ] && echo "OK:
bundle_fp==covered"

# 2) Build a passing manifest + verify
printf
'{"artifacts":[{"name":"response.json","bytes":%s,"sha256":"%s","content_type":"application/json"},
{"name":"receipt.json","bytes":%s,"sha256":"%s","content_type":"application/json"},{"name":"trace.
json","bytes":%s,"sha256":"%s","content_type":"application/json"}]}\n' \
  "$(wc -c < response.json)" "$(sha256sum response.json | awk '{print $1}')" \
  "$(wc -c < receipt.json)"  "$(sha256sum receipt.json  | awk '{print $1}')" \
  "$(wc -c < trace.json)"    "$(sha256sum trace.json    | awk '{print $1}')" \
> manifest.json; \
curl -sS -X POST "$HOST/v3/verify/upload" \
  -F 'response_json=@response.json;type=application/json' \
  -F 'receipt_json=@receipt.json;type=application/json' \
  -F 'trace_json=@trace.json;type=application/json' \
  -F 'bundle_manifest_json=@manifest.json;type=application/json' \
| jq '{pass, checks:(.checks|map({(.name):.ok})|add)}'
```

Integrity Proof

bundle_fp:
sha256:8426d5ed423149665f5786b274cf8176807a493603860a755822012a68d1c6e0
covered:
sha256:8426d5ed423149665f5786b274cf8176807a493603860a755822012a68d1c6e0

Minimal manifest (display only)


```
{"artifacts":[{"name":"trace.json","bytes":986,"sha256":"2c
70cce815cd4e9f2f7d786e528a5ba65e26eef7feaee56a540f8defc57d1
113","content_type":"application/json"}]}
```


Verify Summary (Key Checks)


```
{
  "pass": true,
  "checks": {
    "bundle_schema": true,
    "policy_fp_applied": true,
    "bundle_inventory": true,
    "signature_verify": true,
    "manifest_integrity": true,
    "edge_schema": true,
    "allowed_ids_subset": true
  }
```


Fail-closed (no snapshot)

```
HTTP/1.1 412 Precondition Failed
date: Sun, 02 Nov 2025 17:52:23 GMT
server: uvicorn
content-type: application/json
content-length: 33
x-trace-id: b4a19b4c5bc67824d4....
x-request-id: 73a59e17d667471a

{"detail":"precondition:missing"}
```

Short Answer including citations

BatVault Memory

Enter a decision reference.

CEO | Manager | Analyst

Adopt stripe for eu billing

Trace

SHORT ANSWER

Sam Rivera on 2025-08-15: Adopt Stripe for EU billing.

Select and integrate Stripe to support EU billing and VAT handling for the Q4 launch

Key events

- PCI SAQ-A scope confirmed
- Load tests meet SLOs for billing service

Next: Approve EU billing production rollout

Owner: Sam Rivera (VP Engineering) ID: eng#d-eng-010

eng#d-eng-010 eng#e-eng-002 eng#e-eng-001 eng#d-eng-011

Expand Audit

Signature and hash verification

BatVault Memory

Enter a decision reference.

CEO | Manager | Analyst

Adopt stripe for eu billing

SHORT ANSWER

Sam Rivera on 2025-08-15: Adopt Stripe for EU billing.

Select and integrate Stripe to support EU billing and VAT handling for the Q4 launch

Key events

- PCI SAQ-A scope confirmed
- Load tests meet SLOs for billing service

Next: Approve EU billing production rollout

Owner: Sam Rivera (VP Engineering) ID: eng#d-eng-010

eng#d-eng-010 eng#e-eng-002 eng#e-eng-001 eng#d-eng-011

Audit

Verification result

ed25519 | offline-capable | deterministic

✅ Verified — Signature valid; all fingerprints match.

The receipt signs `bundle_fp`, a deterministic hash of the full response. We recompute fingerprints locally and compare them to the server's claim.

Request ID `8e6ec0f68ddf4b10` Copy Signed at `2025-11-01T23:51:51Z` Key ID `gateway/k1`

`bundle_fp` signature target	Gateway `sha256…d1c6e0` Copy / Local `sha256…d1c6e0` Copy	✓
`graph_fp` derived	Gateway `sha256…10831a` Copy / Local `sha256…10831a` Copy	✓
`allowed_ids_fp` derived	Gateway `sha256…bb5f1d` Copy / Local `sha256…bb5f1d` Copy	✓
`receipt.covered` receipt	Gateway `sha256…d1c6e0` Copy / Local `sha256…d1c6e0` Copy	✓
`policy_fp` inside bundle	Gateway `sha256…805014` Copy / Local `sha256…805014` Copy	✓
`schema_fp` header	Gateway –	—

Receipt

Download receipt.json Copy

▼ Show receipt JSON

This receipt is an Ed25519 signature over `bundle_fp`. Anyone with the public key can verify it offline.


```
{
  "alg": "ed25519",
  "covered": "sha256:8426d5ed423149665f5786b274cf8176807a493603860a755822012a68d1c6e0",
  "key_id": "gateway/k1",
  "sig": "SNDbu1enkLtCRnB40Z6qSo+Hu8QP/8ZogApx98jSejxO1U8MpMWDIAv0u0tVcOJNkELMQ1Q62DfV661U6WsSCw=
  "signed_at": "2025-11-01T23:51:51Z"
```


Re-verify | View bundle | Download full | Receipt

Expand (including mask summaries) and graph view

ALL ALLOWED IDS (4)

Close

EVENT | ALIAS EVENT | sales#e-sales-alias-eng-010

Collapse

RFP requires EU billing and VAT

2025-08-18T09:00:00Z

Wire (masked node only) | Item (masked + mask summary)


```
{
  "id": "sales#e-sales-alias-eng-010",
  "type": "EVENT",
  "domain": "sales",
  "title": "RFP requires EU billing and VAT",
  "description": "Top prospect requires EU billing support in Q4; deal is blocked until available.",
  "timestamp": "2025-08-18T09:00:00Z",
  "mask_summary": {
    "total_removed": 5,
    "items": [
      {
        "field": "x-extra.anchor",
        "reason_code": "policy:field_denied"
      },
      {
        "field": "x-extra.decision_ref",
        "reason_code": "policy:field_denied"
      },
      {
        "field": "x-extra.prospect",
        "reason_code": "policy:field_denied"
      },
      {
        "field": "x-extra.sensitivity",
        "reason_code": "policy:field_denied"
      },
      {
        "field": "x-extra.value_eur",
        "reason_code": "policy:field_denied"
      }
    ]
  }
}
```


EVENT | eng#e-eng-001

Details

Load tests meet SLOs for billing service

2025-08-12T16:00:00Z

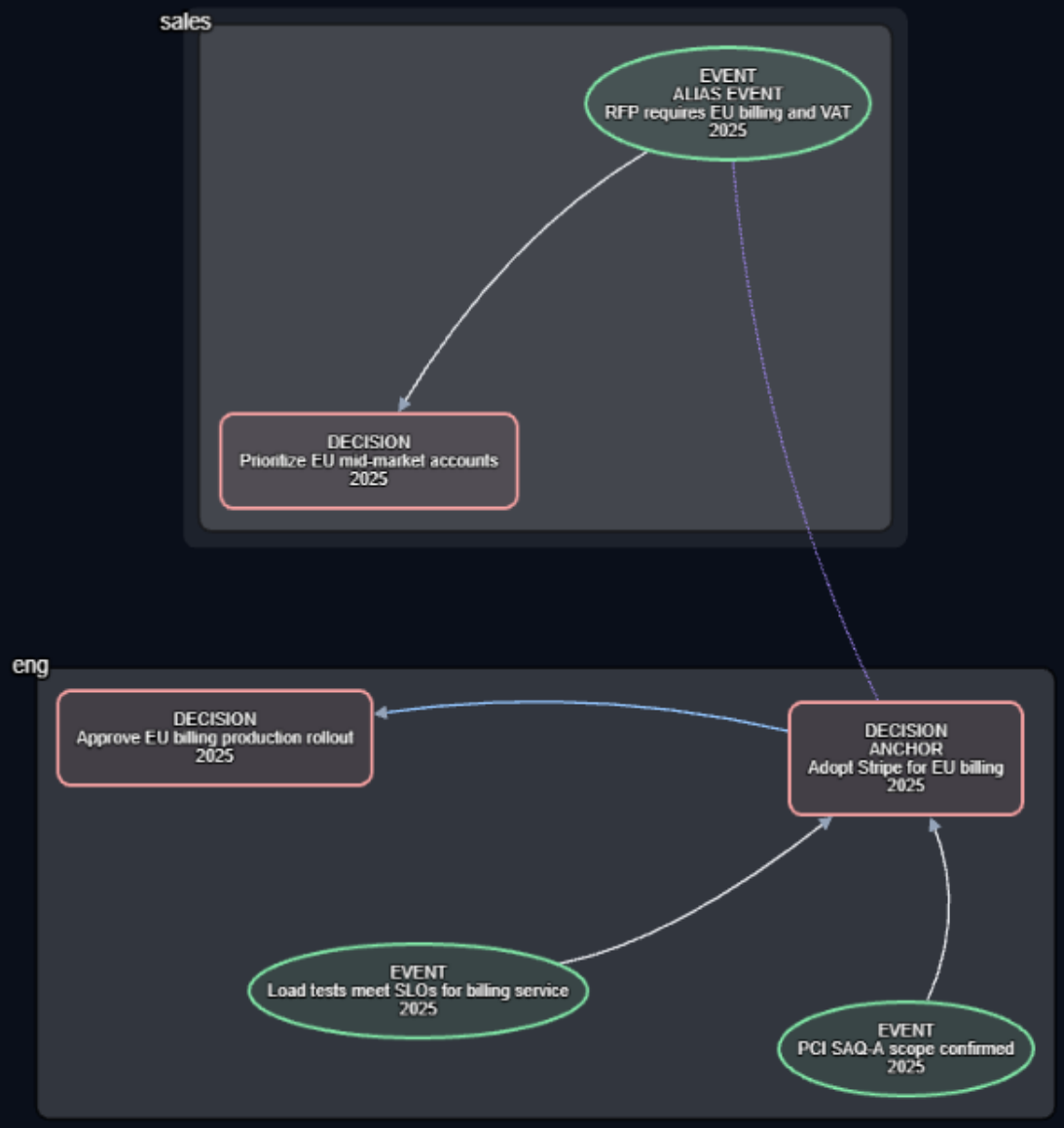
DECISION | sales#d-sales-005

Details

Prioritize EU mid-market accounts

2025-08-20T14:00:00Z

DECISION | ANCHOR | eng#d-eng-010

Details

Adopt Stripe for EU billing